

Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen¹

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway¹

April 30

© 1999-2025, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week of April 28-May 2

1. Classical Support Vector Machines, reminder from last week
2. Quantum Support Vector Machines
3. And if we get time, we may start with Quantum Neural Networks

Reading recommendations

1. For classical Support Vector Machines, see lecture notes by Morten Hjorth-Jensen at https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/chapter5.html
2. Claudio Conti, Quantum Machine Learning (Springer), sections 1.5-1.12, see <https://link.springer.com/book/10.1007/978-3-031-44226-1>. With an IP-number from the university of Oslo, you can download this book for free. The material on quantum support vector machines will be discussed next week.

What is Machine Learning?

Machine Learning (ML) is the study of algorithms that improve through data experience.

Types of Machine Learning:

1. **Supervised Learning:** Labeled data for classification or regression.
2. **Unsupervised Learning:** No labels; discover hidden patterns.
3. **Reinforcement Learning:** Learning through interaction with the environment.

ML Workflow:

Data → Model Training → Prediction

What is Quantum Machine Learning?

Quantum Machine Learning (QML) integrates quantum computing with machine learning algorithms to exploit quantum advantages.

Motivation:

1. High-dimensional Hilbert spaces for better feature representation.
2. Quantum parallelism for faster computation.
3. Quantum entanglement for richer data encoding.

Quantum Speedups in ML

Why Quantum?

1. **Quantum Parallelism:** Process multiple states simultaneously.
2. **Quantum Entanglement:** Correlated states for richer information.
3. **Quantum Interference:** Constructive and destructive interference to enhance solutions.

Example - Quantum Fourier transforms:

$$\text{DiscreteFourierTransform} \sim O(N^2)$$

$$\text{FastFourierTransform} \sim O(N^2)$$

$$\text{QuantumFourierTransform} \sim O(N)$$

Challenges in Quantum Machine Learning

Quantum Hardware Limitations:

1. Noisy Intermediate-Scale Quantum (NISQ) devices.
2. Decoherence and limited qubit coherence times.

Data Encoding:

1. Efficient embedding of classical data into quantum states.

Scalability:

1. Difficult to scale circuits to large datasets.

Classical Support Vector Machines (SVMs)

Support Vector Machines (SVM) are supervised learning algorithms used for classification tasks, and regression tasks as well. The main goal of SVMs is to find the optimal separating hyperplane (in high-dimensional space) that provides a maximum margin between classes.

First mathematical formulation of SVMs (formal details below)

For a dataset (x_i, y_i) where $x_i \in \mathbb{R}^n$ and $y_i \in \{-1, 1\}$, the decision boundary is defined as:

$$f(x) = w \cdot x + b = 0.$$

The goal is to optimize

$$\min_{w,b} \frac{1}{2} \|w\|^2,$$

subject to

$$y_i(w \cdot x_i + b) \geq 1, \quad \forall i$$

Kernels and more

In classical SVMs, kernels help with non-linear data separations. Quantum computers can speed up the computation of complex kernel evaluations by efficiently simulating an inner product in an exponentially large Hilbert space.

Quantum Support Vector Machines (QSVMs)

Quantum Kernel Estimation:

1. Maps classical data to a quantum Hilbert space.
2. Quantum kernel measures similarity in high-dimensional space.

Quantum-enhanced kernel:

$$K(\mathbf{x}, \mathbf{x}') = |\langle \psi(\mathbf{x}) | \psi(\mathbf{x}') \rangle|^2$$

Here, $|\phi(\mathbf{x})\rangle$ is the quantum state encoding of the classical data $\mathbf{x}$.
Advantage: Potentially exponential speedup over classical SVMs.

Quantum Neural Networks (QNNs)

Quantum Neural Networks replace classical neurons with parameterized quantum circuits.

Key Concepts:

1. Quantum Gates as Activation Functions.
2. Variational Quantum Circuits (VQCs) for optimization.

Parameterized Quantum Circuit:

$$U(\theta) = \prod_i R_y(\theta_i) \cdot CNOT \cdot R_x(\theta_i)$$

Advantage: Quantum gradients enable exploration of non-convex landscapes.

Quantum Boltzmann Machines (QBMs)

Quantum Boltzmann Machines leverage quantum mechanics to sample from a probability distribution.

1. Quantum tunneling aids in escaping local minima.
2. Quantum annealing for optimization problems.

Quantum Hamiltonian:

$$H = - \sum_i b_i \sigma_i^z - \sum_{ij} w_{ij} \sigma_i^z \sigma_j^z$$

Advantage: Efficient sampling in complex probability distributions.

Quantum SVMs

The idea of a classical SVM is that we have a set of points that are in either one group or another and we want to find a line that separates these two groups. This line can be linear, but it can also be much more complex, which can be achieved by the use of Kernels.

What will we need in the case of a quantum computer?

We will have to translate the classical data point $\vec{x}$ into a quantum datapoint $|\Phi(\vec{x})\rangle$. This can be achieved by a circuit $\mathcal{U}_{\Phi(\vec{x})}|0\rangle$. Here $\Phi()$ could be any classical function applied on the classical data $\vec{x}$.

We need a parameterized quantum circuit $W(\theta)$ that processes the data in a way that in the end we can apply a measurement that returns a classical value -1 or 1 for each classical input $\vec{x}$ that identifies the label of the classical data.

The most general ansatz

Following these steps we can define an ansatz for this kind of problem which is

$$W(\theta)\mathcal{U}_{\Phi}(\vec{x})|0\rangle.$$

These kind of ansatzes are called quantum variational circuits.

Quantum SVM

In the case of a quantum SVM we will only use the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})},$$

to translate the classical data into quantum states and build the Kernel of the SVM out of these quantum states. After calculating the Kernel matrix on the quantum computer we can train the Quantum SVM the same way as the classical SVM.

Defining the Quantum Kernel

The idea of the quantum kernel is exactly the same as in the classical case. We take the inner product

$$K(\vec{x}, \vec{z}) = |\langle \Phi(\vec{x}) | \Phi(\vec{z}) \rangle|^2 = \langle 0^n | \mathcal{U}_{\Phi(\vec{x})}^\dagger \mathcal{U}_{\Phi(\vec{z})} | 0^n \rangle,$$

but now with the quantum feature maps

$$\mathcal{U}_{\Phi(\vec{x})}.$$

The idea is that if we choose a quantum feature maps that is not easy to simulate with a classical computer we might obtain a quantum advantage.

Side note

There is no proof yet that the QSVM brings a quantum advantage, but the argument the authors of <https://arxiv.org/pdf/1804.11326.pdf> make, is that there is for sure no advantage if we use feature maps that are easy to simulate classically, because then we would not need a quantum computer to construct the Kernel.

Feature map

For the feature maps we use the ansatz

$$\mathcal{U}_{\Phi(x)} = U_{\Phi(x)} \otimes H^{\otimes n},$$

where

$$U_{\Phi(x)} = \exp \left(i \sum_{S \in n} \phi_S(x) \prod_{i \in S} Z_i \right),$$

which simplifies a lot when we (like in

<https://arxiv.org/pdf/1804.11326.pdf>) only consider $S \leq 2$ interactions, which means we only let two qubits interact at a time. For $S \leq 2$ the product $\prod_{i \in S}$ only leads to interactions $Z_i Z_j$ and non interacting terms Z_i . And the sum $\sum_{S \in n}$ over all these terms that are possible with n qubits.

Classical functions

Finally we define the classical functions $\phi_i(\vec{x}) = x_i$ and

$$\phi_{i,j}(\vec{x}) = (\pi - x_i)(\pi - x_j).$$

If we write this ansatz for 2 qubits and $S \leq 2$ we see how it simplifies:

$$U_{\Phi(x)} = \exp(i(x_1 Z_1 + x_2 Z_2 + (\pi - x_1)(\pi - x_2) Z_1 Z_2)).$$

We won't get into details too much here, why we would take this ansatz. It is simply an ansatz that is simple enough and leads to good results.

Finally we can define a depth of these circuits. Depth 2 means we repeat this ansatz two times. Which means our feature map becomes $U_{\Phi(x)} \otimes H^{\otimes n} \otimes U_{\Phi(x)} \otimes H^{\otimes n}$

Plans for next week

1. Introducing quantum neural networks